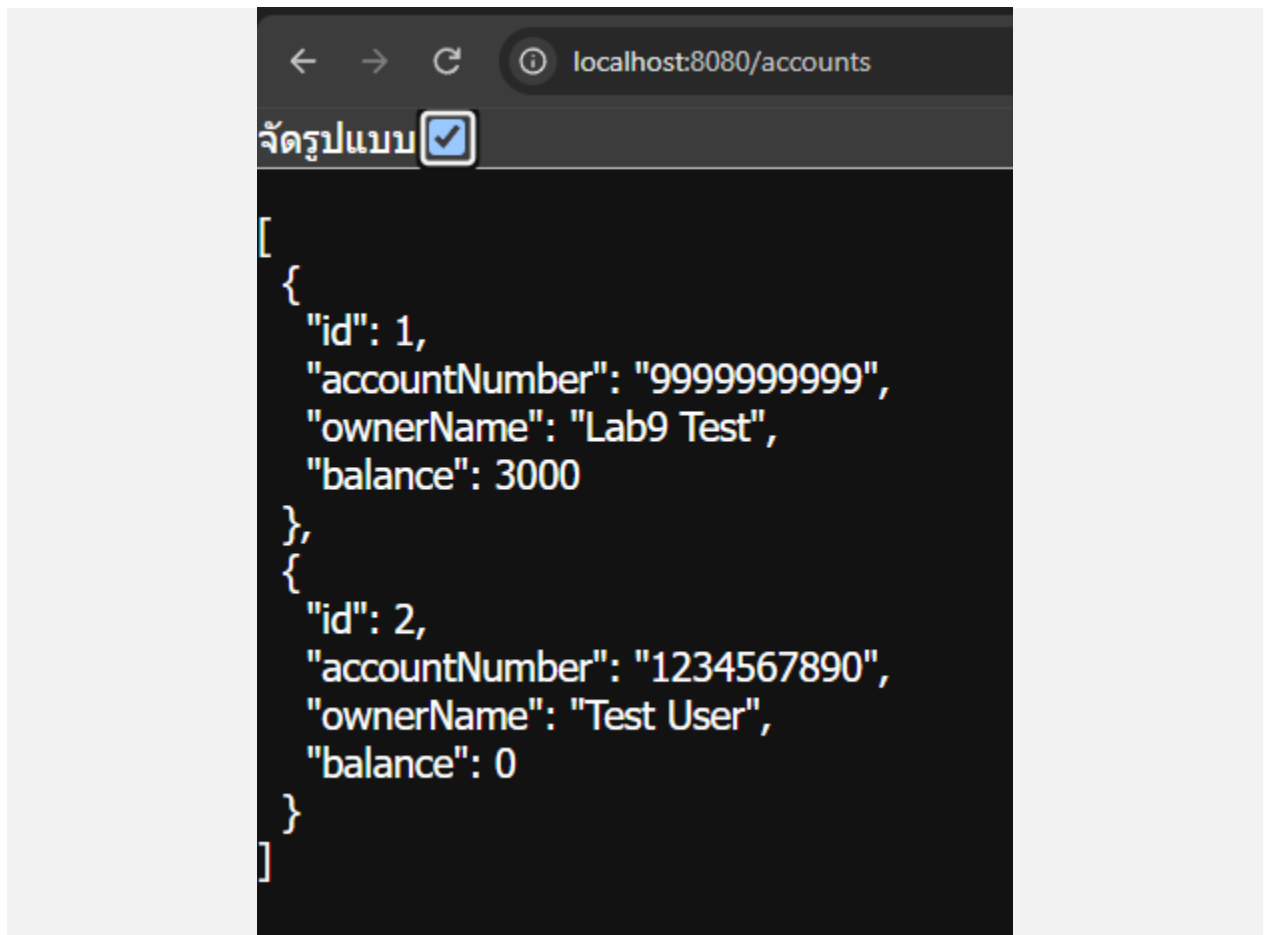


Lab 9: Spring Boot Transaction

ชื่อ-นามสกุล: นายลลิตศักดิ์ ภูขยัน

รหัสนักศึกษา: 673380035-5

1. การสร้าง Account



```
[
  {
    "id": 1,
    "accountNumber": "9999999999",
    "ownerName": "Lab9 Test",
    "balance": 3000
  },
  {
    "id": 2,
    "accountNumber": "1234567890",
    "ownerName": "Test User",
    "balance": 0
  }
]
```

อธิบายผล: ระบบสามารถสร้างบัญชีธนาคารได้สำเร็จ โดยบันทึกเลขบัญชี ชื่อเจ้าของบัญชี และยอดเงินเริ่มต้นลงในฐานข้อมูล พร้อมสร้าง id สำหรับอ้างอิงบัญชีในขั้นตอนถัดไป

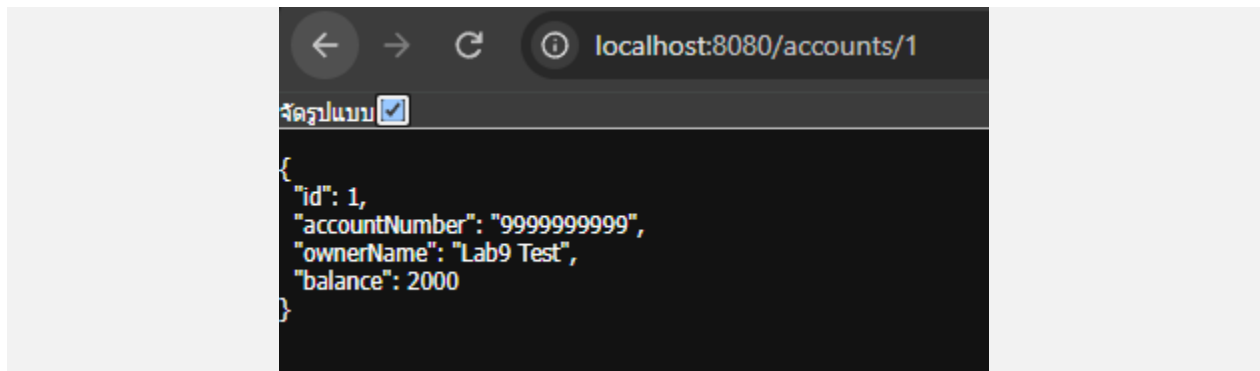
2. การฝากเงินสำเร็จ

```
PS C:\lab9-transaction> Invoke-RestMethod -Method Post -Uri "http://localhost:8080/accounts/1/deposit" -ContentType "application/json" -Body '{"amount":1000}'

message
-----
Deposit successful
```

อธิบายผล: ระบบรับค่าขอฝากเงินจำนวน 1,000 บาทและตอบกลับข้อความ Deposit successful แสดงว่าการฝากเงินทำงานสำเร็จ

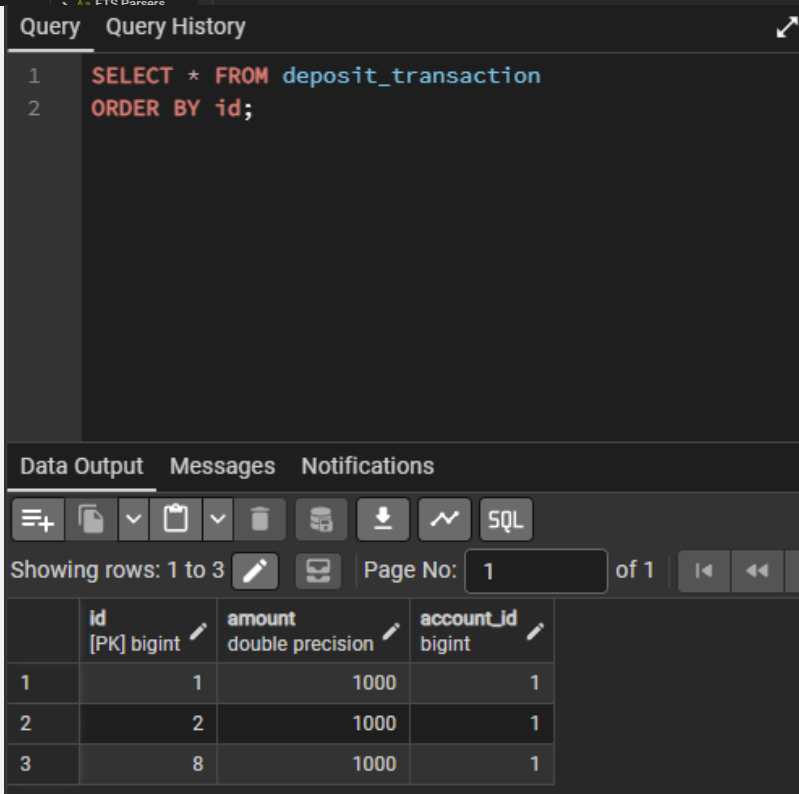
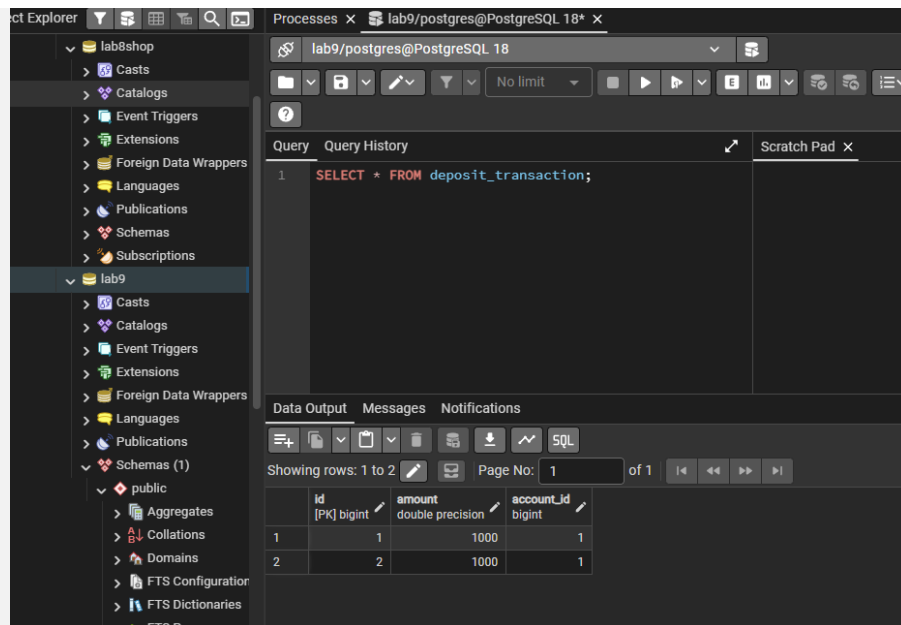
3. ตรวจสอบ Account



```
{
  "id": 1,
  "accountNumber": "9999999999",
  "ownerName": "Lab9 Test",
  "balance": 2000
}
```

อธิบายผล: เมื่อตรวจสอบบัญชีหลังฝากเงิน พบว่ายอด balance เพิ่มขึ้นจากยอดเดิมเป็น 3,000 บาทตามจำนวนรายการฝากที่ทดสอบ แสดงว่าการแก้ไขยอดเงินใน Account ทำงานถูกต้อง

4. ตรวจสอบ DepositTransaction

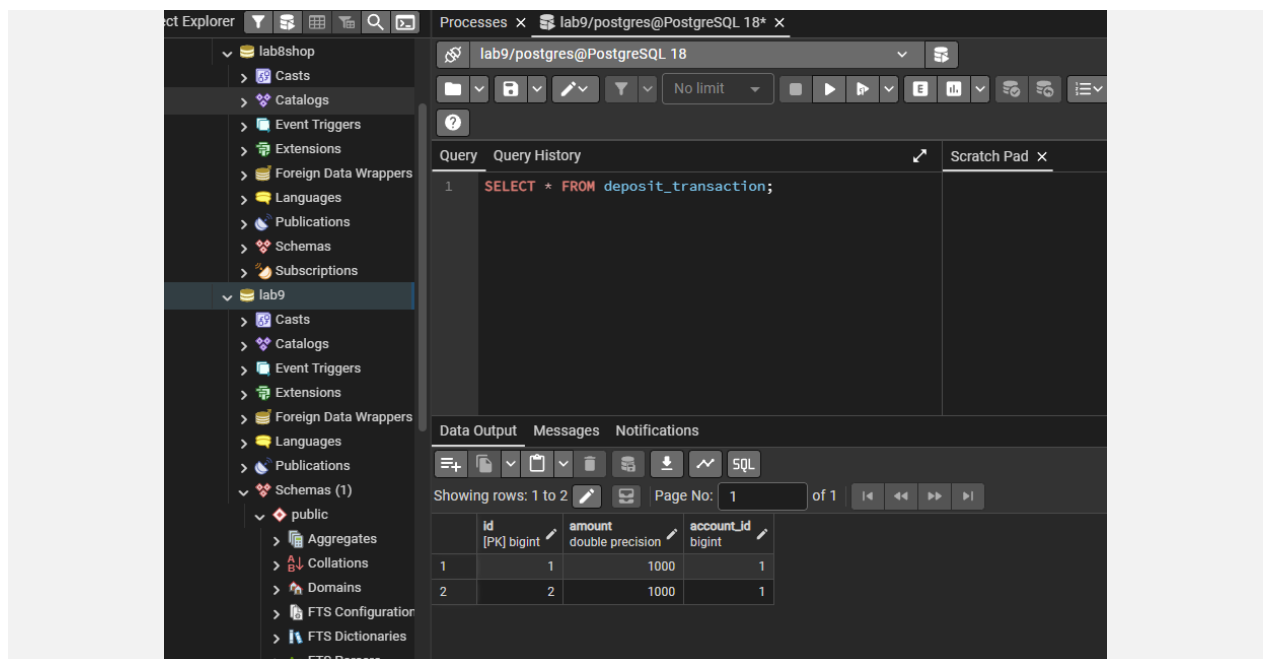
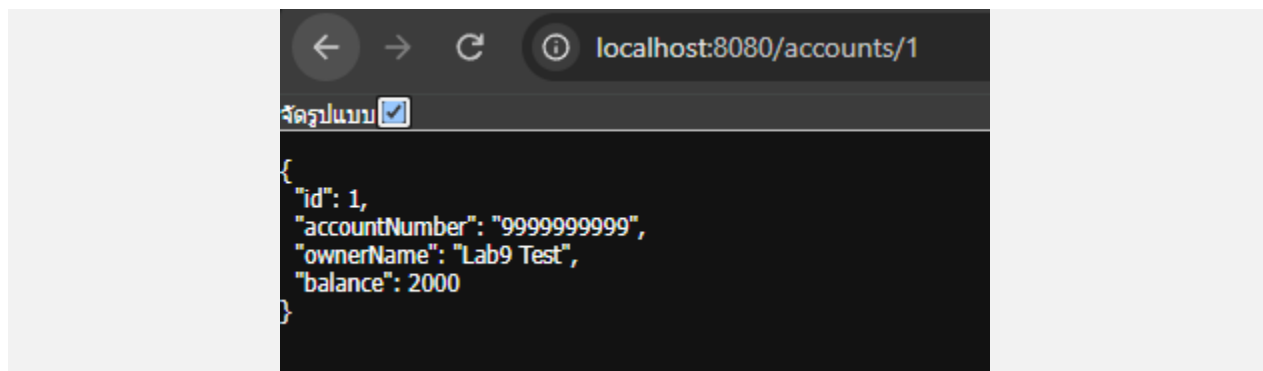


อธิบายผล: ตาราง deposit_transaction แสดงรายการฝากเงินที่ถูกบันทึก โดยมีคอลัมน์ id, amount และ account_id แต่ละรายการ

เชื่อมโยงกับ Account ผ่าน account_id แสดงว่าระบบบันทึกประวัติการฝากเงินสำเร็จ

5. ทดลอง Rollback

```
PS C:\WINDOWS\system32> try { Invoke-WebRequest -Method Post -Uri "http://localhost:8080/accounts/1/deposit" -ContentType "application/json" -Body '{"amount":1000}' -ErrorAction Stop } catch { $_.Exception.Response.StatusCode.value__ }
500
```



อธิบายผล: เมื่อเปิดใช้ @Transactional และเกิด RuntimeException หลังจากบันทึก Account และ DepositTransaction แล้ว Spring จะสั่ง ROLLBACK ทำให้การเปลี่ยนแปลงทั้งหมดของการฝากครั้งนั้นถูกยกเลิก ยอดเงินและประวัติการฝากจึงไม่เพิ่มขึ้น

6. เปรียบเทียบมี @Transactional กับไม่มี @Transactional

```
PS C:\WINDOWS\system32> try { Invoke-WebRequest -Method Post -Uri "http://localhost:8080/accounts/1/deposit" -ContentType "application/json" -Body '{"amount":1000}' -ErrorAction Stop } catch { $_.Exception.Response.StatusCode.value__ }
500
```

```
src > main > java > com > example > lab9 > service > J DepositService.java > DepositService
1  package com.example.lab9.service;
2
3  import com.example.lab9.model.Account;
4  import com.example.lab9.model.DepositTransaction;
5  import com.example.lab9.repository.AccountRepository;
6  import com.example.lab9.repository.DepositRepository;
7  import org.springframework.stereotype.Service;
8  import org.springframework.transaction.annotation.Transactional;
9
10 @Service
11 public class DepositService {
12
13     private final AccountRepository accountRepository;
14     private final DepositRepository depositRepository;
15
16     public DepositService(AccountRepository accountRepository,
17                           DepositRepository depositRepository) {
18         this.accountRepository = accountRepository;
19         this.depositRepository = depositRepository;
20     }
21
22     public void deposit(Long accountId, Double amount) {
23
24         if (amount == null || amount <= 0) {
25             throw new IllegalArgumentException(s: "Deposit amount must be greater than 0");
26         }
27
28         Account account = accountRepository.findById(accountId)
29             .orElseThrow(() ->
30                 new RuntimeException("Account not found: " + accountId));
31
32         double currentBalance = account.getBalance() == null ? 0.0 : account.getBalance();
33         account.setBalance(currentBalance + amount);
34         accountRepository.save(account);
35
36         DepositTransaction deposit = new DepositTransaction();
37         deposit.setAmount(amount);
38         deposit.setAccount(account);
39         depositRepository.save(deposit);
40
41         // ใช้สำหรับข้อทดลอง Rollback เท่านั้น:
42         throw new RuntimeException(message: "Test Rollback");
43     }
44 }
45
```

```
localhost:8080/accounts/1

จัดรูปแบบ ☒

{
  "id": 1,
  "accountNumber": "9999999999",
  "ownerName": "Lab9 Test",
  "balance": 3000
}
```

Query Query History

1

2

```
SELECT * FROM deposit_transaction
ORDER BY id;
```

Data Output Messages Notifications

≡+

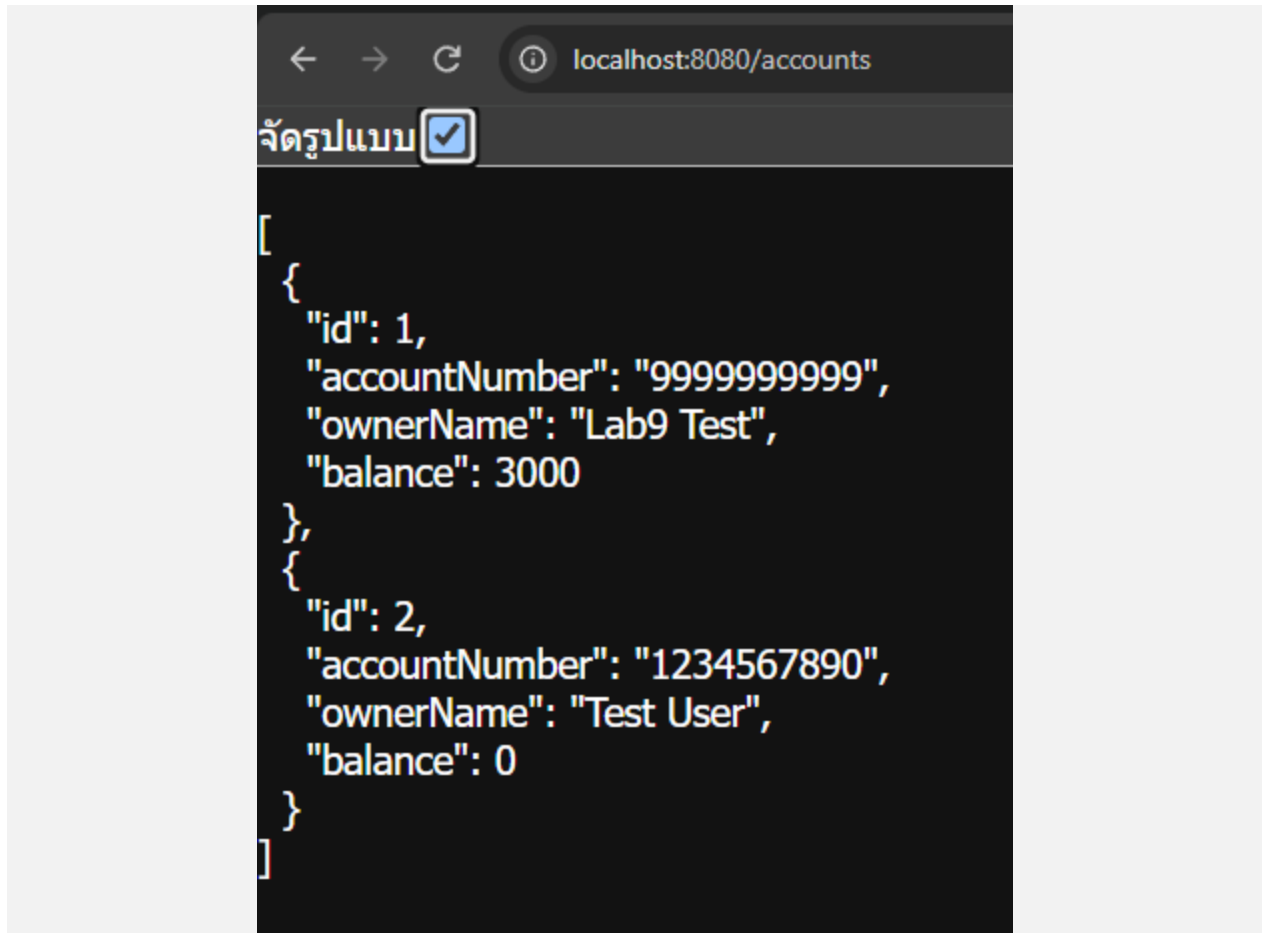
▼

▼

SQL

Showing rows: 1 to 3 Page No: 1 of 1

	id [PK] bigint	amount double precision	account_id bigint
1	1	1000	1
2	2	1000	1
3	8	1000	1



```
[
  {
    "id": 1,
    "accountNumber": "9999999999",
    "ownerName": "Lab9 Test",
    "balance": 3000
  },
  {
    "id": 2,
    "accountNumber": "1234567890",
    "ownerName": "Test User",
    "balance": 0
  }
]
```

อธิบายผล:

มี @Transactional	ไม่มี @Transactional
เกิด Error แล้ว Rollback ทั้งหมด	เกิด Error แต่ข้อมูลที่ save ก่อนหน้า อาจยังคงอยู่
ยอดเงินไม่เพิ่ม	ยอดเงินอาจเพิ่ม เช่น จาก 2,000 เป็น 3,000
ประวัติฝากไม่เพิ่ม	ประวัติฝากอาจเพิ่มค้างในฐานข้อมูล

7. คำตอบคำถามท้าย Lab

1. @Transactional มีหน้าที่อะไร?

@Transactional กำหนดให้คำสั่งหลายขั้นตอนทำงานอยู่ใน Transaction เดียวกัน ถ้าทุกขั้นตอนสำเร็จจึง COMMIT แต่ถ้าเกิด RuntimeException จะ ROLLBACK การเปลี่ยนแปลงทั้งหมด

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

การฝากเงินต้องทำสองอย่างคือเพิ่ม balance ใน Account และบันทึก DepositTransaction ทั้งสองอย่างควรสำเร็จพร้อมกัน เพื่อป้องกันยอดเงินกับประวัติการฝากไม่ตรงกัน

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

COMMIT คือการยืนยันการเปลี่ยนแปลงให้ถูกบันทึกถาวรในฐานข้อมูล ส่วน ROLLBACK คือการยกเลิกการเปลี่ยนแปลงใน Transaction และคืนข้อมูลกลับไปก่อนเริ่มงาน

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

เป็นความสัมพันธ์แบบ One-to-Many โดย Account หนึ่งบัญชีมี DepositTransaction ได้หลายรายการ และ DepositTransaction แต่ละรายการอ้างอิง Account หนึ่งบัญชีผ่าน account_id

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ถ้าเกิด RuntimeException Spring จะสั่ง ROLLBACK ทำให้การเพิ่ม balance และการบันทึก DepositTransaction ของการฝากครั้งนั้นถูกยกเลิกทั้งหมด ข้อมูลในฐานข้อมูลจึงไม่เปลี่ยนแปลง

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

เมื่อไม่มี @Transactional คำสั่งบันทึกอาจถูกยืนยันแยกกัน เมื่อเกิด Error ภายหลังข้อมูลที่บันทึกไปก่อนหน้านี้อาจยังคงอยู่ ทำให้ balance เพิ่มขึ้นแต่เกิด Error และประวัติการฝากอาจถูกบันทึกค้าง